OPERATING SYSTEM — SESSIONAL 2 CHEAT SHEET

Priority: ★★★★★ = sure-shot / numerical · ★★★★ = very likely · ★★★ = likely (short Q) · ★★ = MCQ/definition · ★ = read once | **Formulas:** TAT = CT − AT · WT = TAT − BT · RT = first-CPU − AT · Avg = Σ/n · Need = Max − Allocation

1. PRIORITY SCHEDULING ★★★★★

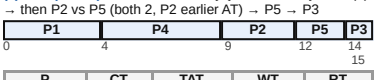
CPU given to process with **highest priority** (assume small number = high priority unless Q says otherwise). Tie → **FCFS** (earlier AT).

- **Non-preemptive:** once running, runs till completion; new high-priority job waits in ready queue at head.
- **Preemptive:** new arrival with higher priority **preempts** running process. Check at every arrival.
- **Starvation / Indefinite blocking:** low-priority process may wait forever.
- **Aging** (solution): gradually **increase priority** of waiting process (e.g. +1 every 15 min / factor 0.5 per day).
- Adv: important/system processes first. Disadv: starvation; equal priority needs another algo.

Numerical (lower no. = higher priority)

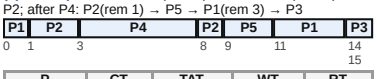
P	AT	BT	Pr
P1	0	4	3
P2	1	3	2
P3	2	1	4
P4	3	5	1
P5	4	2	2

(a) Non-preemptive — at t=4 ready {P2,P3,P4,P5} → P4(1) → then P2 vs P5 (both 2, P2 earlier AT) → P5 → P3



P	CT	TAT	WT	RT
P1	4	4	0	0
P2	12	11	8	8
P3	15	13	12	12
P4	9	6	1	1
P5	14	10	8	8
Avg		**8.8**	**5.8**	**5.8**

(b) Preemptive — t=1 P2 preempts P1; t=3 P4 preempts P2; after P4: P2(rem 1) → P5 → P1(rem 3) → P3



P	CT	TAT	WT	RT
P1	14	14	10	0
P2	9	8	5	0
P3	15	13	12	12
P4	8	5	0	0
P5	11	7	5	5
Avg		**9.4**	**6.4**	**3.4**

Keywords: preemption, starvation, aging, indefinite blocking, static vs dynamic priority

2. ROUND ROBIN (RR) ★★★★★

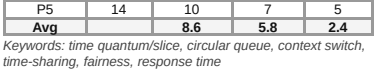
FCFS + **preemption** using fixed **time quantum (q)**; designed for **time-sharing**. Ready queue is **circular FIFO**.

- Process runs min(q, remaining); if unfinished → goes to **tail** of queue.
- **Rule:** processes that **arrive during** a slice enter queue **before** the preempted process.
- q very large → behaves like **FCFS**; q very small → too many **context switches** (overhead).
- Max wait for a process ≤ (n−1)·q. Good **response time**.
- Adv: **no starvation**, fair. Disadv: high context-switch overhead, avg WT often higher than SJF.

Numerical (q = 2)

P	P1	P2	P3	P4	P5
AT	0	1	2	3	4
BT	5	3	1	2	3

Queue trace: [P1] → t2: P2,P3,P1 → t4: P3,P1,P4,P5,P2 → t5: P1,P4,P5,P2 → t7: P4,P5,P2,P1 → t9: P5,P2,P1 → t11: P2,P1,P5 → t12: P1,P5 → t13: P5



P	CT	TAT	WT	RT
P1	13	13	8	0
P2	12	11	8	1
P3	5	3	2	2
P4	9	6	4	4
P5	14	10	7	5
Avg		**8.6**	**5.8**	**2.4**

Keywords: time quantum/slice, circular queue, context switch, time-sharing, fairness, response time

3. MULTILEVEL QUEUE (MLQ) ★★★

- Ready queue split into **separate queues**: foreground (interactive) → **RR**; background (batch) → **FCFS**.
- Process **permanently assigned** to a queue — **no movement** between queues.
- Between queues: **fixed priority** (starvation possible) or **time slice** (e.g. 80% fg / 20% bg).
- Order: System → Interactive → Interactive editing → Batch → Student processes.

4. MULTILEVEL FEEDBACK QUEUE (MLFQ) ★★★★

- Processes **can move** between queues → uses **aging** to prevent starvation.
- Parameters: no. of queues, algo per queue, when to **upgrade**, when to **demote**, which queue a new process enters.
- **Example:** Q0 RR q=8 → Q1 RR q=16 → Q2 FCFS. New job enters Q0; not finished in 8 ms → demoted to Q1; not finished in 16 more → Q2.
- Adv: most flexible, low response time, prevents starvation. Disadv: **complex**, CPU overhead, tuning parameters.

Algo	Preemptive?	Starvation	Basis
FCFS	No	No (convoy)	AT
SJF	No	Yes	BT
SRTF	Yes	Yes	rem. BT
Priority	Both	Yes→aging	Priority
RR	Yes	No	Quantum
MLQ	Both	Yes	Fixed queue
MLFQ	Yes	No (aging)	Feedback

5. PROCESS vs THREAD ★★★★

Thread = basic unit of CPU utilization = **lightweight process**; has own **PC, stack, registers, thread ID**; shares **code, data, files, heap** (memory) of its process.

Process	Thread
Heavyweight, own address space	Lightweight, shares address space
Creation/switch costly	Cheap create & switch
IPC needed to communicate	Communicate via shared memory
Isolated – one crash ≠ others	One thread fault can kill process
Own PCB	TCB inside process

Thread States ★★

New (created, not started) → **Runnable/Ready** → **Running** → **Blocked/Waiting/Timed-waiting** (I/O, lock, sleep, join) → back to Runnable → **Terminated/Dead** (cannot restart).

Benefits of Threads ★★★ — "RRES"

- **Responsiveness** – UI continues if part blocked
- **Resource sharing** – share memory by default
- **Economy** – cheaper create/switch than process
- **Scalability** – use multiprocessor/multicore

6. TYPES OF THREADS ★★★

User-Level (ULT)	Kernel-Level (KLT)
Managed by user thread **library**	Managed by **OS kernel**
Fast, no kernel intervention	Slower to create
One blocks → all block	One blocks → others run
No true parallelism on multicore	Scheduled on different CPUs
POSIX Pthreads, Mach C-threads, Solaris threads	Windows NT/2000, Linux, Solaris, Tru64

7. MULTITHREADING MODELS ★★★★

- **Many-to-One:** many ULT → 1 KLT. One blocking call blocks all; no parallelism. Ex: Solaris Green Threads, GNU Portable Threads.
- **One-to-One:** each ULT → own KLT. More concurrency, parallel on multicore; overhead of creating KLT. Ex: Windows, Linux, OS/2.
- **Many-to-Many:** m ULT → ≤ m KLT (dynamic mapping). Best of both: many threads + parallelism, no all-block. Ex: Solaris (pre-9), Windows ThreadFiber.

8. HYPER-THREADING (HT) ★★★

- Intel technology; a form of **SMT (Simultaneous Multithreading)**.
- 1 **physical core** appears as **2 logical processors** to OS.
- Each logical CPU has own **architectural state (registers)**; shares **execution units, cache**.
- When one thread **stalls** (cache miss/memory wait), other uses idle execution units.
- Benefits: ↑ throughput, better resource utilization, cheap parallelism. Best for mixed compute + memory/I-O workloads. **Not** 2× speed.

9. CASE STUDY: RTOS SCHEDULING ★★

- **Hard RT** (missed deadline = failure: ABS, pacemaker) vs **Soft RT** (degraded quality: video streaming).
- RTOS traits: **determinism, predictability, low latency**; preemptive priority-based. Ex: FreeRTOS, VxWorks, QNX, RT-Linux, Zephyr.
- **RMS** (Rate Monotonic): static, shorter **period** = higher priority. Test: $U = \Sigma Ci/Pi \le n(2^{1/n}-1)$ (n=2 → 0.828).
- **EDF**: dynamic, earliest **absolute deadline** first; optimal, schedulable if $U \le 1$.
- **DMS**: static, shorter **relative deadline** = higher priority.
- **Priority inversion**: high task blocked by low task holding resource → fix with **Priority Inheritance (PIP)** / **Priority Ceiling (PCP)**. Also: **WCET, jitter, tick rate**.

10. CONCURRENCY & RACE CONDITION ★★★★

- **Concurrency:** multiple processes/threads in **overlapping time** (interleaving is enough, not necessarily simultaneous). Types: process-level, thread-level, distributed.
- **Race condition:** several processes access & manipulate **shared data** concurrently and result **depends on order** of execution.
- `counter++` = reg=counter; reg=reg+1; counter=reg → interleaving with counter-- gives 4 or 6 instead of 5.
- Effects: data inconsistency, non-deterministic bugs. Prevent: mutex/locks, semaphores, atomic ops.

11. CRITICAL SECTION (CS) PROBLEM ★★★★★

CS = code segment accessing shared resources. Structure: `do{ entry section; CS; exit section; remainder section } while(1);`

3 Requirements (MPB):
1. **Mutual Exclusion** – only one process in CS at a time.
2. **Progress** – if CS empty, only processes not in remainder decide who enters next; decision can't be postponed indefinitely.
3. **Bounded Waiting** – limit on how many times others enter CS after a process requests (no starvation).

12. IPC (Inter-Process Communication) ★★★

Needed by **cooperating processes** (info sharing, speedup, modularity, convenience). Two models:

Shared Memory	Message Passing
Region of memory shared	send(msg)/receive(msg)
Faster (kernel only at setup)	Slower (system call each msg)
Large data	Small data, distributed systems
Programmer handles sync	Easier, OS handles

- **Direct:** send(P,msg), receive(Q,msg) – explicit naming, link per pair.
- **Indirect:** via **mailbox/port**: send(A,msg), receive(A,msg) – link shared by many.

13. LOCK VARIABLE ★★★★★

Simplest **software**, busy-wait mechanism. Shared `int lock=0` (0 = free, 1 = busy). Works for **n** processes, user mode.

```
int lock = 0;              // shared
while(1){
  while(lock == 1);        // entry: busy wait
  lock = 1;                // acquire
  /* Critical Section */
  lock = 0;                // exit: release
  /* Remainder */ }
```

× Fails Mutual Exclusion!
P1 reads lock=0 → preempted before setting 1 → P2 reads 0, sets 1, enters CS → P1 resumes, sets 1, also enters CS. Reason: **test & set are 2 separate (non-atomic) steps**. Also busy waiting. ✓ Progress × Bounded waiting. Fix → **Test-and-Set**, mutex, semaphore.

Strict Alternation (turn variable) ★★★

```
int turn=0;
P0: while(turn!=0); CS; turn=1;
P1: while(turn!=1); CS; turn=0;
```

✓ ME × **Progress** (P0 can't re-enter if P1 not interested) – strict P0,P1,P0… only 2 processes, busy wait.

Mutex Lock

`acquire(){ while(!available); available=false; }` `release(){ available=true; }` – must be atomic; busy-waiting version = **spinlock**.

14. PETERSON'S SOLUTION ★★★★

Software solution for **2 processes**; shared `int turn; bool flag[2];`

```
// Process Pi (j = 1-i)
do{ flag[i]=true;  turn=j;
    while(flag[j] && turn==j);  // wait
    /* CS */
    flag[i]=false;
    /* remainder */ } while(1);
```

✓ ME ✓ Progress ✓ Bounded waiting. × only 2 processes, busy waiting, may fail on modern CPUs (instruction reordering).

Bakery Algorithm (Lamport) ★★

For **n** processes. Take ticket `number[i]=max(number)+1`; lowest ticket enters; tie → lower PID: `(a,i)<(b,j)`. Uses `choosing[]`, `number[]`. FCFS, no special hardware; busy wait.

Test-and-Set (TSL) – Hardware ★★★

```
bool TestAndSet(bool *t){   // ATOMIC
  bool rv=*t; *t=true; return rv; }
do{ while(TestAndSet(&lock)); // spin
    /* CS */  lock=false; }while(1);
```

Check + set in **one indivisible instruction** → ✓ ME. × busy wait (spinlock), × bounded waiting not guaranteed.

15. SEMAPHORES ★★★★

Integer **S** accessed only via 2 **atomic** ops (Dijkstra):

```
wait(S)   /P/down: while(S<=0); S--;
signal(S) /V/up:   S++;
```

- **Binary** (0/1) ≈ mutex lock. **Counting** (any int) – for resources with n instances.
- Ordering: S1 before S2 → `synch=0`; P1: S1; signal(synch); P2: wait(synch); S2.
- No-busy-wait impl.: **block()** into waiting queue, **wakeup()** on signal. Negative S = no. of waiting processes.
- Misuse → **deadlock & starvation**: signal…wait swapped, wait…wait, omitting wait/signal.

16. BOUNDED BUFFER (Producer–Consumer) ★★★★★

Producer puts items into buffer of size **n**, consumer removes. Producer must wait if **full**; consumer waits if **empty**; no simultaneous buffer access.

Semaphore	Init	Purpose
mutex	1	mutual exclusion on buffer
empty	**n**	count of empty slots
full	**0**	count of filled slots

```
Producer:                       Consumer:
do{ produce item;               do{ wait(full);
  wait(empty);                    wait(mutex);
  wait(mutex);                    remove item;
  add item to buffer;             signal(mutex);
  signal(mutex);                  signal(empty);
  signal(full);                   consume item;
}while(1);                      }while(1);
```

⚠ **Swapping wait(empty)/wait(mutex) → DEADLOCK (producer holds mutex while buffer full).**

Without semaphores (counter++/counter--) → race condition. Unbounded buffer: no size limit.

17. READERS–WRITERS ★★★★★

Shared data: **many readers** may read together; a **writer needs exclusive** access (no reader/writer with it).
Shared: `semaphore rw_mutex=1, mutex=1; int read_count=0;`

```
Writer:                         Reader:
do{                             do{ wait(mutex);
  wait(rw_mutex);                   read_count++;
  /* writing */                     if(read_count==1)
  signal(rw_mutex);                   wait(rw_mutex);
}while(1);                          signal(mutex);
                                    /* reading */
                                    wait(mutex);
                                    read_count--;
                                    if(read_count==0)
                                       signal(rw_mutex);
                                    signal(mutex);
                                  }while(1);
```

- **First reader** locks out writers, **last reader** releases. `mutex` protects read_count.
- **1st R-W problem (Reader preference)** – above code → **writers may starve**.
- **2nd R-W problem (Writer preference)** – once a writer waits, no new reader enters → readers may starve.

18. DINING PHILOSOPHERS ★★★★★★

5 philosophers, round table, 1 bowl of rice, **5 chopsticks** (one between each pair). Think ↔ eat; need **both left & right** chopsticks. `semaphore chopstick[5] = {1,1,1,1,1};`

```
do{ wait(chopstick[i]);
    wait(chopstick[(i+1)%5]);
    /* eat */
    signal(chopstick[i]);
    signal(chopstick[(i+1)%5]);
    /* think */ }while(1);
```

Problem: all 5 pick left chopstick simultaneously → all wait for right → DEADLOCK (circular wait). Also starvation.
Remedies: (1) allow at most **4** philosophers at table; (2) pick up only if **both** available (inside CS); (3) **asymmetric**: odd → left then right, even → right then left; (4) monitor solution with states THINKING/HUNGRY/EATING.

Sleeping Barber ★★

1 barber, 1 barber chair, N waiting chairs. No customer → barber sleeps; customer wakes him; all chairs full → customer leaves. Semaphores: `customers=0, barber=0, mutex=1, int waiting=0.`

```
Barber: while(1){ wait(customers);
  wait(mutex); waiting--; signal(barber);
  signal(mutex); cut_hair(); }
Customer: wait(mutex);
  if(waiting<N){ waiting++; signal(customers);
    signal(mutex); wait(barber); get_cut(); }
  else signal(mutex);          // leave
```

19. DEADLOCK ★★★★★

Set of processes each **waiting for a resource held by another** in the set → none proceeds. Resource use: **request → use → release**.

4 Necessary Conditions (Coffman) – ALL must hold:
1. **Mutual Exclusion** – resource non-sharable.
2. **Hold & Wait** – holding ≥1 and waiting for more.
3. **No Preemption** – released only voluntarily.
4. **Circular Wait** – P0→P1→…→Pn→P0.

Resource Allocation Graph (RAG)

- Circle = process, box with dots = resource instances. **Request edge** Pi→Rj, **Assignment edge** Rj→Pi, **Claim edge** (dashed) Pi⇢Rj.
- No cycle ⇒ no deadlock. Cycle + **single instance** per type ⇒ **deadlock**. Cycle + **multiple instances** ⇒ **possibility** only.

Methods of Handling

Prevention (break a condition) · **Avoidance** (never enter unsafe state) · **Detection & Recovery** · **Ignore** (Ostrich algo – Windows/UNIX).

Prevention ★★★★

- **ME:** make resources sharable (read-only files) – not possible for printer.
- **Hold&Wait:** request all resources before start, or request only when holding none → low utilization, starvation.
- **No Preemption:** if request can't be granted, release all held resources; restart later.
- **Circular Wait:** impose **total ordering** of resource types; request in increasing order. (most practical)

Avoidance

Each process declares **max** need a priori. **Safe state**: exists a **safe sequence** <P1..Pn> where each Pi's need ≤ Available + held by all Pj (j<i). Safe ⇒ no deadlock; Unsafe ⇒ possible deadlock (deadlock ⊂ unsafe). Single instance → **RAG with claim edges** (grant only if no cycle); multiple instances → **Banker's**.

Detection ★★★

- Single instance: **wait-for graph** (remove resource nodes; Pi→Pj if Pi waits for Pj). Cycle ⇒ deadlock. O(n²).
- Multiple instances: like safety algo using **Request** matrix (Finish[i]=true if $Allocation_i$=0). Any Finish[i]=false ⇒ Pi deadlocked.

Recovery

Process termination: abort all, or one at a time (choose by priority, time computed, resources used/needed, interactive/batch). **Resource preemption:** select victim (min cost) → **rollback** → avoid **starvation** (count rollbacks in cost).

Case study: Oracle / MySQL InnoDB — T1 locks row 101 waits 202, T2 locks 202 waits 101 → circular wait; DB detects via wait-for graph, rolls back victim (ERROR 1213 / ORA-00060).

20. BANKER'S ALGORITHM ★★★★★

Deadlock **avoidance**, multiple instances. n processes, m resource types.
Available[m], Max[n][m], Allocation[n][m], Need = Max − Allocation. Total = Available + ΣAllocation.

Safety Algorithm

1) Work=Available; Finish[i]=false ∀i. 2) find i: Finish[i]==false && $Need_i \le Work$; none → step 4. 3) **Work = Work + $Allocation_i$**; Finish[i]=true; go 2. 4) all Finish true ⇒ **SAFE**. Complexity O(m·n²).

Resource-Request Algorithm

1) $Request_i \le Need_i$? else **error** (exceeds max). 2) $Request_i \le$ Available? else **wait**. 3) Pretend: Available −= Req; $Alloc_i$ += Req; $Need_i$ −= Req → run safety: safe ⇒ **grant**, unsafe ⇒ restore & wait.

Numerical 1 (A=10, B=5, C=7)

P	Alloc ABC	Max ABC	Need ABC
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = **(3,3,2)** [= 10,5,7 − ΣAlloc 7,2,5]

Step	Pick	Need ≤ Work?	New Work
1	P1	122 ≤ 332 ✓	5 3 2
2	P3	011 ≤ 532 ✓	7 4 3
3	P0	743 ≤ 743 ✓	7 5 3
4	P2	600 ≤ 753 ✓	10 5 5
5	P4	431 ≤ 1055 ✓	10 5 7

✅ **SAFE**: <P1, P3, P0, P2, P4> — other valid orders (e.g. <P1,P3,P4,P2,P0>) also get full marks.

Request P1 (1,0,2): ≤ Need(1,2,2) ✓, ≤ Avail(3,3,2) ✓ → Avail=(2,3,0), Alloc P1=(3,0,2), Need P1=(0,2,0). Safety → <P1,P3,P4,P0,P2> safe ⇒ **GRANT**. (Then P4 req (3,3,0) ⇒ > Avail → wait; P0 req (0,2,0) ⇒ unsafe → deny.)

Numerical 2 (X Y Z), Available = (1,1,2)

P	Alloc	Max	Need
P0	2 1 1	5 1 3	3 0 2
P1	1 0 1	2 1 2	1 1 1
P2	4 1 2	7 2 3	3 1 1
P3	1 0 0	2 1 1	1 1 1

Work 112 → P1 (111≤112) → **213** → P3 (111≤213) → **313** → P0 (302≤313) → **524** → P2 (111≤524) → **936**.
✅ **SAFE**: <P1, P3, P0, P2>. Check: final Work = Total = (9,3,6).

Keywords: a-priori max claim, safe sequence, Need ≤ Work, pretend allocation, Work += Allocation

⚡ EXAM QUICK-HITS ★★★★

- Dispatcher gives CPU to process chosen by **short-term scheduler**; dispatch latency.
- Semaphore is an **integer variable**; two kinds: **binary & counting**.
- Bounded buffer = **Producer–Consumer**. Circular wait prevented by **linear ordering** of resources.
- Dining problem occurs with **5 philosophers, 5 chopsticks**. Writers get **exclusive** access.
- Wait-for graph Pi→Pj ⇒ Pi waiting for Pj; applies to **single-instance** resources.
- Algorithm with no ME guarantee: **Round Robin** (it's scheduling, not sync).
- Thread shares **memory space** with parent; owns PC, registers, stack.
- HT duplicates **registers** per core → **2 logical processors**.
- RTOS requirement: **predictable timing**. MLFQ uses **dynamic feedback**.
- Busy waiting = **spinlock**; wastes CPU but no context switch (good on multicore, short waits).